



CAHIER DES CHARGES

Fonctionnel et Technique

**Développement d'un système mobile "Offline-First"
et d'un moteur de formulaires dynamiques pour
l'audit des infrastructures IT**

Réalisé par :

Hanae OUKACHA

Elève Ingénieure en Ingénierie Logicielle

Entreprise d'accueil :

Substancium Business Services (SBS) –Rabat Agdal

Établissement d'origine :

École Nationale Supérieure de l'Intelligence Artificielle et Science des Données

Encadrant en entreprise : M. Mourad

Encadrant académique : Pr. Loubna KARBIL

Année universitaire : 2025-2026

Introduction Générale

Dans un contexte technologique où la disponibilité et la sécurité des données sont devenues des enjeux vitaux, le maintien en condition opérationnelle des infrastructures informatiques critiques (data centers, serveurs, baies de brassage) est une priorité absolue. C'est au cœur de cette exigence que s'inscrit le présent projet, réalisé dans le cadre de mon stage au sein de **Substancium Business Services (SBS)**, un acteur historique et majeur de l'infogérance et de l'intégration technologique au Maroc.

Afin d'anticiper les défaillances matérielles chez ses clients, SBS déploie une stratégie rigoureuse de maintenance préventive. Cependant, le processus de validation de ces audits sur le terrain fait face à des limites structurelles importantes :

- **Une dépendance totale au format papier** : Les techniciens utilisent des checklists physiques spécifiques à chaque marché pour valider l'état des équipements.
- **Une charge administrative lourde** : La nécessité d'une ressaisie manuelle des données en fin de journée génère une perte de temps considérable et un risque élevé d'erreurs de transcription.
- **Un manque d'agilité** : La mise à jour des questions d'audit nécessite la réimpression de nouveaux formulaires, ralentissant l'adaptation aux nouvelles normes des clients.

Pour répondre à cette problématique opérationnelle, ce cahier des charges propose la conception et le développement d'une **solution logicielle de numérisation intégrale**. L'objectif est de déployer un écosystème centralisé composé d'un portail web de supervision et d'une application mobile dédiée aux techniciens. Afin de s'adapter aux contraintes physiques des salles de serveurs, cette application fonctionnera selon une architecture "Offline-First" et s'appuiera sur un moteur de formulaires dynamiques générés à la volée depuis des schémas de données (JSON Schemas).

CHAPITRE I : CADRE ET CONTEXTE DU PROJET

I. Présentation de l'entreprise : Substancium Business Services

1. Un acteur historique du paysage informatique marocain

Créée en 1978, Substancium s'impose comme une entreprise pionnière et un acteur majeur de l'intégration technologique au Maroc. Disposant de son siège social à Casablanca et de bureaux opérationnels à Rabat, l'entreprise a largement contribué à façonner le secteur des technologies de l'information dans le royaume. Historiquement, Substancium a été à l'avant-garde des grands chantiers de transformation numérique du pays, notamment en menant les premiers projets de migration vers des systèmes ouverts pour l'Administration publique. L'entreprise s'est également illustrée en prenant en charge le déploiement informatique de vastes réseaux d'agences bancaires et postales, tout en introduisant très tôt des concepts novateurs sur le marché local, comme l'intégration de grands progiciels de gestion (ERP).

2. Domaines d'expertise et d'intervention

Aujourd'hui, le cœur de métier de Substancium s'articule autour du conseil, de l'infogérance, et du maintien en condition opérationnelle des infrastructures informatiques et digitales. Certifiée ISO 9001:2015, l'entreprise accompagne ses clients dans la définition d'architectures techniques complexes, le déploiement de solutions applicatives, et la gestion d'infrastructures électriques sécurisées telles que les solutions d'onduleurs. Son offre couvre l'ensemble de la chaîne de valeur, allant de la maintenance décentralisée des postes de travail utilisateurs jusqu'à la supervision globale de data centers et de réseaux intelligents pour des clients d'envergure, incluant des ministères et de grands opérateurs de télécommunications.

II. Contexte du projet :

Dans le cadre de son activité d'infogérance et de support technique, Substancium est responsable de la santé matérielle et logicielle des infrastructures IT critiques de ses clients. Pour prévenir les pannes matérielles, anticiper les défaillances des serveurs et garantir la continuité des services, l'entreprise déploie une stratégie rigoureuse de maintenance préventive. Cette stratégie se traduit par des tournées d'inspection régulières sur les sites d'exploitation. Les techniciens de Substancium effectuent ainsi des contrôles périodiques, qu'ils soient trimestriels, semestriels ou annuels, afin d'auditer l'état global du parc matériel informatique, électrique et réseau.

Lors de ces interventions sur site, les techniciens doivent s'assurer que chaque équipement respecte des normes strictes de sécurité, d'environnement et de performance. Pour guider ces audits complexes, ils s'appuient sur des protocoles d'inspection détaillés se présentant sous forme de checklists. Ces formulaires répertorient l'ensemble des points de contrôle visuels et techniques à valider pour chaque catégorie spécifique d'équipement, qu'il s'agisse de serveurs de données, de baies de brassage, ou de systèmes de refroidissement. À l'heure actuelle, ce processus fondamental d'audit et de validation repose entièrement sur un support physique traditionnel.

III. Problématique opérationnelle :

Bien que l'expertise technique des intervenants sur le terrain soit reconnue, le processus actuel de collecte et de traitement des données d'inspection souffre de faiblesses structurelles majeures liées à l'usage exclusif du format papier. Sur les différents sites, les techniciens remplissent manuellement leurs formulaires de contrôle. Une fois leurs tournées achevées, ces documents physiques sont rapatriés aux bureaux. Les employés administratifs sont alors chargés de déchiffrer les annotations manuscrites pour les ressaisir intégralement dans le système de l'entreprise. Cette procédure de double saisie est non seulement extrêmement chronophage, mais elle engendre également un goulot d'étranglement administratif lourd à la fin de chaque journée d'intervention.

Au-delà de la simple perte de temps de traitement, cette méthode de travail archaïque expose Substancium à des risques opérationnels non négligeables. La manipulation constante de feuilles volantes dans des environnements techniques parfois étroits augmente considérablement le risque de perte, de dégradation ou d'oubli de documents de certification. De plus, la ressaisie humaine constitue une source inévitable d'erreurs de frappe ou de mauvaise interprétation des notes. Ce décalage temporel entre le constat physique sur le terrain et l'enregistrement numérique de l'information crée également un manque total de traçabilité en temps réel. Si un technicien identifie une faille critique de sécurité ou une panne imminente sur un serveur le matin, l'alerte n'est numérisée et visible par les superviseurs qu'en toute fin de journée, retardant d'autant la prise de décision et l'intervention d'urgence.

Enfin, le système actuel s'avère particulièrement rigide et inadapté face aux évolutions technologiques constantes de l'IT. Lorsqu'un nouveau type de matériel est intégré au parc informatique d'un client, ou qu'un nouveau marché impose la modification des normes d'audit, les équipes de Substancium doivent reconcevoir, imprimer et redistribuer physiquement de

nouvelles liasses de checklists à l'ensemble des techniciens mobiles. Cette incapacité à mettre à jour les formulaires d'inspection de manière centralisée, instantanée et dynamique freine drastiquement l'agilité de l'entreprise et complique son adaptation rapide aux nouveaux besoins opérationnels.

IV. Solution proposée

Pour pallier les limites opérationnelles du processus actuel, le présent projet vise à concevoir et développer une solution de numérisation intégrale du flux de maintenance préventive. Cette solution s'articule autour d'une architecture centralisée et bidirectionnelle, scindée en deux portails distincts mais étroitement interconnectés :

- **Un portail Web de supervision (Back-office) :** Destiné aux encadrants et superviseurs de Substancium, cette plateforme centralisée fera office de tour de contrôle. Elle permettra la gestion granulaire des marchés, l'administration de l'inventaire des sites, la planification précise des interventions, ainsi que le suivi analytique des données remontées du terrain.
- **Une application mobile de terrain (Front-office) :** Destinée exclusivement aux techniciens, cette application servira de terminal de saisie et de consultation. Consciente des réalités physiques des infrastructures IT (data centers, salles de serveurs) souvent dépourvues de connectivité réseau, l'application fonctionnera selon un paradigme strictement "Offline-First". Les techniciens pourront ainsi réaliser leurs audits en mode déconnecté, sans aucune interruption de service.
- **Un moteur de formulaires dynamiques :** L'innovation majeure de la solution réside dans l'abandon des interfaces codées en dur. Le système s'appuiera sur des modèles de données au format JSON (JSON Schema) gérés depuis le serveur. L'application mobile interprétera ces schémas pour générer dynamiquement l'interface utilisateur (champs de texte, cases à cocher, listes déroulantes) selon le contexte spécifique du marché et de l'équipement audité.

Cette approche garantit à Substancium une flexibilité totale et une gouvernance stricte des données, supprimant la double saisie tout en offrant une traçabilité quasi instantanée des interventions.

V. Exigences du projet

Afin de répondre efficacement aux problématiques soulevées et de concrétiser la solution proposée, le système à développer devra satisfaire un ensemble d'exigences rigoureuses, réparties en exigences fonctionnelles (liées aux actions des utilisateurs) et non fonctionnelles (liées aux contraintes techniques et qualitatives).

1. Exigences fonctionnelles

a. Portail Web (Superviseur / Administrateur)

- **Gestion des entités métiers** : Le système doit permettre au superviseur de créer, lire, mettre à jour et supprimer (CRUD) les marchés, les sites associés, ainsi que l'inventaire théorique des équipements pour chaque site.
- **Gestion des modèles d'audit** : Le portail doit intégrer une interface permettant d'importer, de valider et d'activer des modèles de checklists au format JSON Schema, avec une gestion stricte du versionnage pour préserver l'historique des formulaires.
- **Planification des missions** : Le superviseur doit pouvoir assigner une mission précise (date, heure, marché, site) à un technicien spécifique via un module de planification (calendrier).
- **Supervision et reporting** : Le système doit offrir un tableau de bord (Dashboard) agrégeant les statistiques des missions (planifiées, en cours, terminées) et permettre l'exportation automatisée des rapports d'intervention sous formats tableurs (Excel) ou documents fixes (PDF).

b. Application Mobile (Technicien)

- **Authentification et synchronisation matinale** : Le technicien doit pouvoir s'authentifier de manière sécurisée et télécharger l'ensemble des données nécessaires à sa journée (planning des missions, inventaire local, modèles JSON) lorsqu'il dispose d'une connexion internet.
- **Exécution hors-ligne (Offline)** : L'application doit permettre la consultation de l'inventaire cible et la génération dynamique des formulaires d'audit sans aucune connexion réseau.

- **Contrôle croisé de l'inventaire** : Lors de la saisie d'un numéro de série ou d'un identifiant matériel, le système doit signaler visuellement (ex: surbrillance verte) sa présence dans l'inventaire théorique.
- **Gestion des écarts matériels** : Le technicien doit avoir la possibilité de déclarer et d'ajouter un équipement non répertorié trouvé sur site, afin de mettre à jour la base de données.
- **Génération de justificatifs locaux** : À la clôture d'une intervention, l'application doit pouvoir générer immédiatement un récapitulatif PDF local si le client sur site l'exige.
- **Synchronisation finale bidirectionnelle** : Dès le rétablissement de la connexion réseau, l'application doit automatiquement transférer les données collectées (interventions, équipements ajoutés) vers le serveur central de manière asynchrone et transparente.

2. Exigences non fonctionnelles

- **Architecture et Disponibilité (Offline-First)** : La continuité de service sur le mobile est critique. La gestion des données locales doit s'appuyer sur un moteur de base de données embarqué robuste (ex: SQLite), capable de stocker temporairement des volumes importants de réponses d'audit sans dégradation des performances.
- **Sécurité et Intégrité des données** : Les échanges entre l'application mobile et l'API centrale doivent être chiffrés et sécurisés via des protocoles d'authentification modernes (ex: tokens JWT). La base de données centrale (ex: PostgreSQL) doit utiliser des types de données adaptés (comme JSONB) pour assurer l'intégrité et l'indexation des réponses dynamiques.
- **Ergonomie et Utilisabilité** : L'interface mobile, développée via des technologies multiplateformes (ex: React Native), doit être intuitive et optimisée pour une utilisation à une main dans des environnements techniques contraignants (faible luminosité, espaces exigus).
- **Évolutivité (Scalabilité)** : L'architecture logicielle, tant au niveau de l'API (FastAPI, Spring Boot ou équivalent) que du modèle relationnel, doit être conçue pour supporter une augmentation significative du nombre de techniciens simultanés et l'ajout futur de nouveaux modèles de formulaires sans nécessiter de refonte du code source.

CHAPITRE II : ÉTUDE CONCEPTUELLE ET CONCEPTION

L'étude conceptuelle constitue une étape charnière du projet. Elle permet de traduire les exigences fonctionnelles et non fonctionnelles en modèles abstraits et rigoureux. Cette transition s'appuie sur le langage de modélisation unifié UML (Unified Modeling Language) afin de structurer les aspects statiques et dynamiques de la solution de maintenance préventive.

I. Diagramme des cas d'utilisation (Use Case)

Le diagramme des cas d'utilisation délimite le périmètre du système en identifiant les interactions entre les différents acteurs (Superviseur, Technicien, Administrateur) et l'application. Il met en évidence la séparation stricte des privilèges entre le personnel de terrain et l'encadrement.

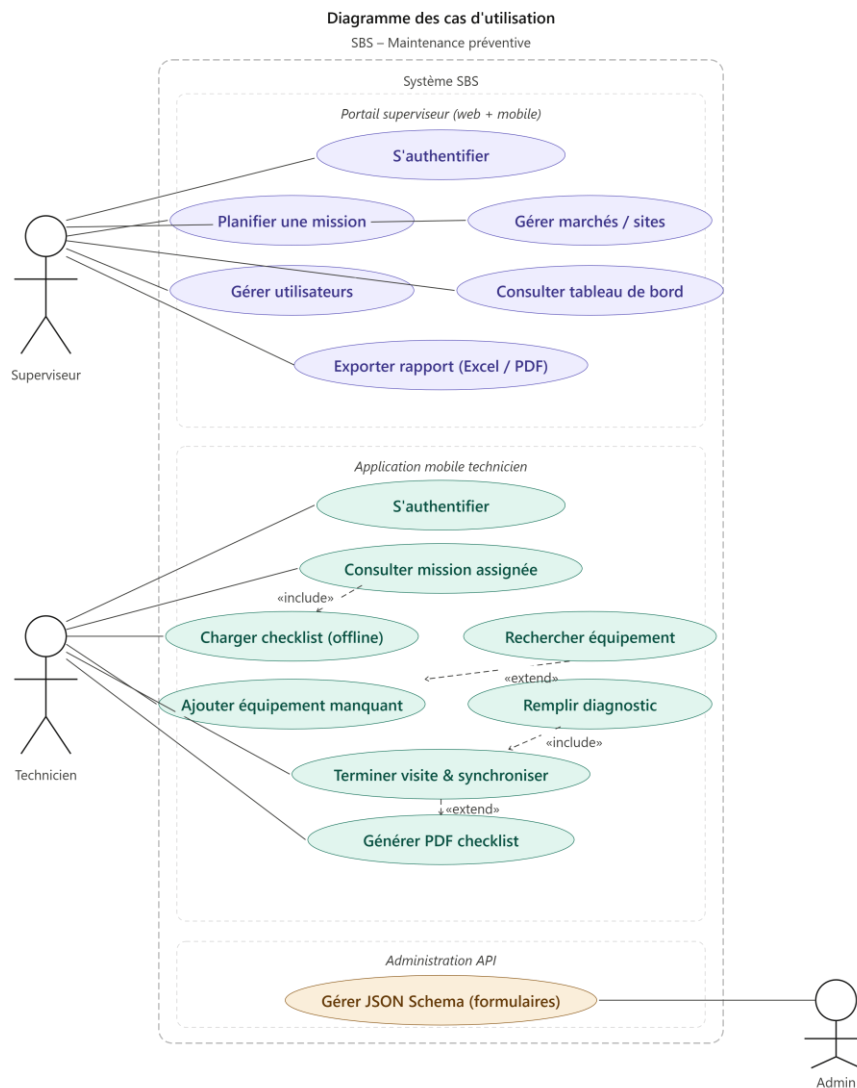


FIGURE 1: DIAGRAMME DES CAS D'UTILISATION DU SYSTEME DE MAINTENANCE PREVENTIVE

II. Diagramme de classes

Le diagramme de classes fournit une vue statique et structurale du système. Il définit les entités métiers fondamentales (Marché, Site, Équipement, Mission, Intervention), leurs attributs, leurs méthodes, ainsi que les contraintes d'association qui les lient, préparant ainsi l'implémentation de la base de données relationnelle et du stockage local.

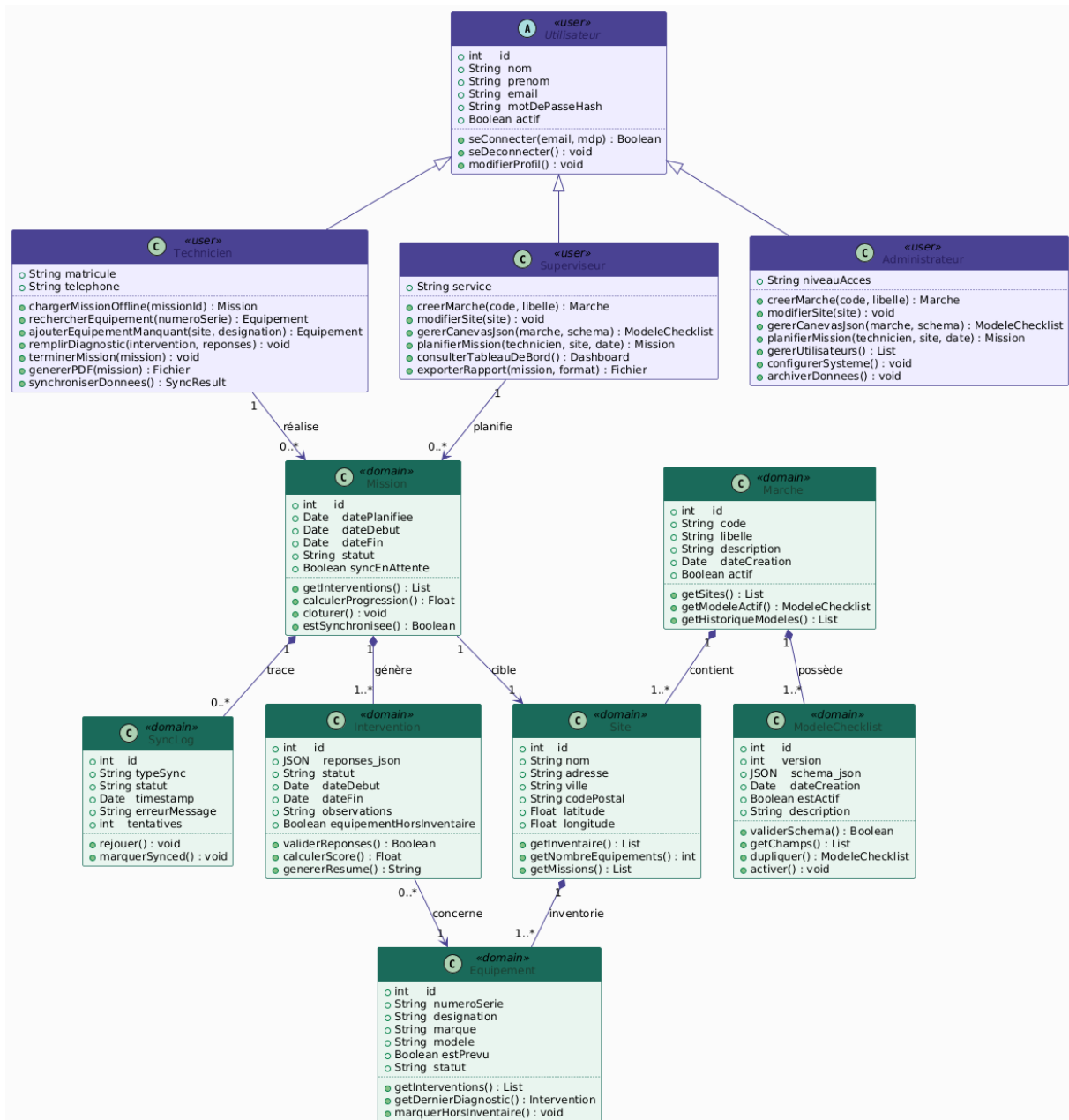


FIGURE 2:DIAGRAMME DE CLASSES DU MODELE DE DONNEES

III. Diagramme de séquence

Le diagramme de séquence décrit la sémantique comportementale et chronologique du système. Il orchestre les échanges de messages entre les acteurs, l'application mobile, l'environnement de stockage local (SQLite) et le serveur central (API et base de données) à travers les différentes phases opérationnelles, de la planification à la synchronisation finale hors-ligne.

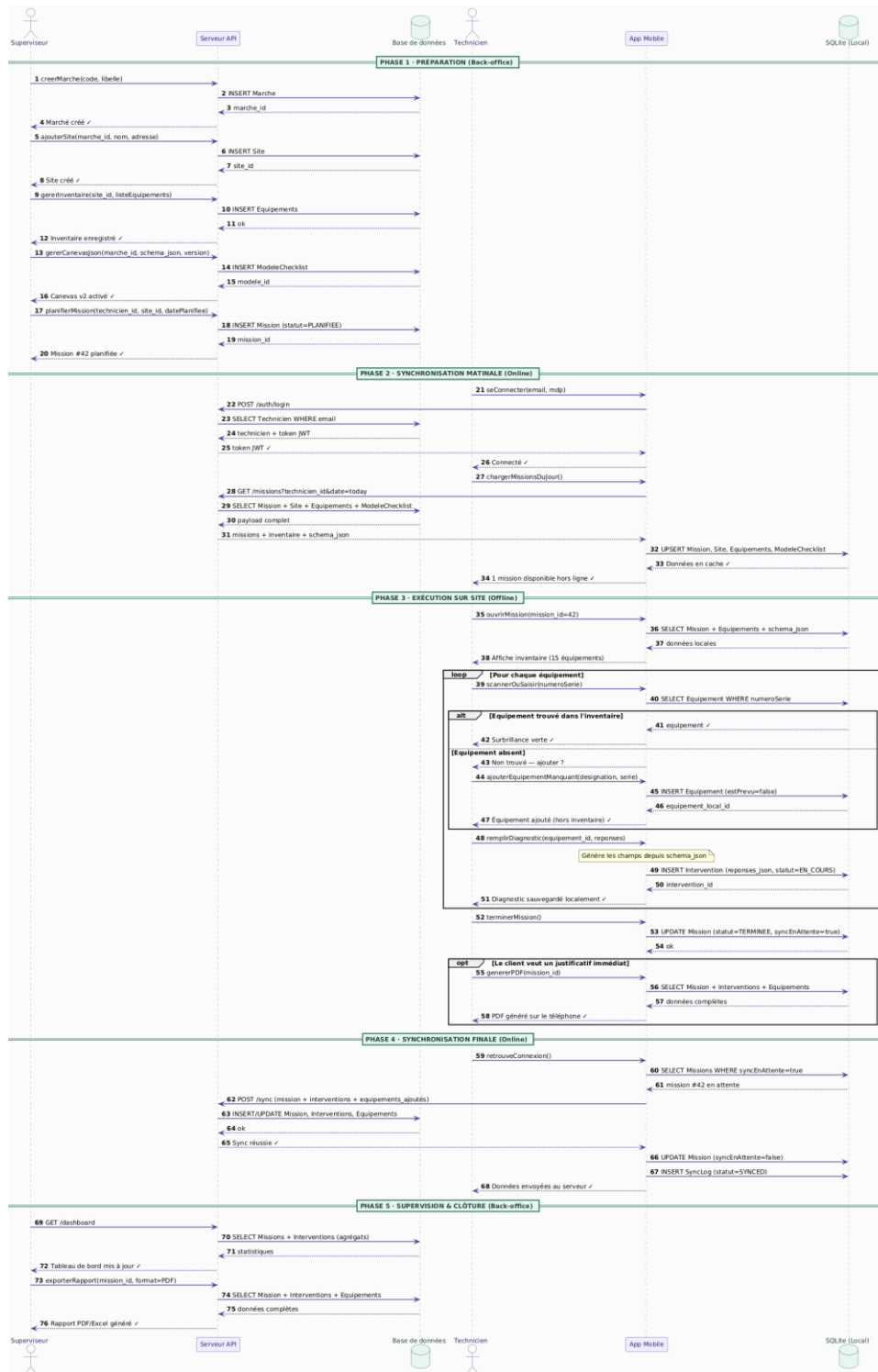


FIGURE 3 DIAGRAMME DE SEQUENCE DU FLUX DE SYNCHRONISATION

CHAPITRE III : ARCHITECTURE ET SPÉCIFICATIONS TECHNIQUES

Ce chapitre définit les fondations techniques de la solution. Il traduit les modèles conceptuels en choix d'infrastructure et de développement, en mettant un accent particulier sur la robustesse du mode déconnecté et la fluidité des échanges de données.

I. Architecture globale du système

Le système s'appuie sur une architecture distribuée à trois niveaux (N-Tier), spécialement adaptée aux contraintes de mobilité. Elle assure une séparation stricte entre la collecte des données sur le terrain, l'orchestration de la logique métier et le stockage centralisé.

1. Schéma de l'architecture applicative

Le diagramme suivant illustre la topologie physique et logique des composants du système, ainsi que les protocoles de communication qui les lient :

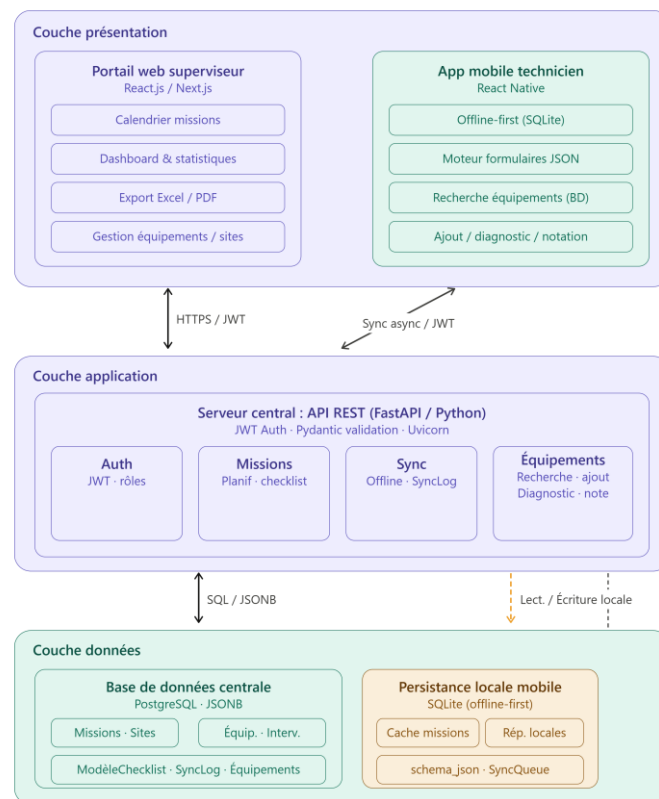


FIGURE 4; SCHEMA DE L'ARCHITECTURE TECHNIQUE GLOBALE DU SYSTEME (N-TIER)

2. Description des composants de l'architecture

- **Le Terminal de Supervision (Interface Web Dashboard) :** Ce composant constitue le point d'entrée pour les administrateurs et les encadrants de Substancium. Il communique directement et exclusivement avec l'API REST centrale via des requêtes HTTPS sécurisées. C'est à travers cette interface que le superviseur gère les référentiels (marchés, sites) et ordonnance le calendrier des missions de maintenance préventive.
- **Le Terminal de Terrain (Application Mobile React Native) :** Déployée sur les smartphones ou tablettes des techniciens, cette application est le pivot de la saisie des audits. Contrairement à une application web classique, elle ne dépend pas d'une connexion permanente à l'API pour fonctionner. Elle interagit directement avec sa propre base de données embarquée pour garantir une disponibilité maximale de l'interface utilisateur.
- **La Base de Données Embarquée (SQLite) :** Ce moteur relationnel léger est directement intégré au stockage local du terminal mobile. Il joue le rôle de mémoire tampon et de cache de haute disponibilité. Il stocke de manière autonome les plannings téléchargés le matin, les structures de formulaires JSON (JSON Schemas) et l'ensemble des réponses d'audits saisies par le technicien tout au long de sa tournée sur site.
- **Le Serveur Applicatif (API REST) :** Ce cœur logique centralise les règles métiers de l'entreprise. L'API expose un ensemble d'endpoints (points d'accès) normalisés qui permettent de recevoir les planifications du portail web, de distribuer les packages de données aux mobiles lors de la synchronisation matinale, et de valider les rapports d'intervention remontés du terrain.
- **La Base de Données Centrale (PostgreSQL) :** Ce système de gestion de base de données (SGBD) centralise l'intégralité du patrimoine informationnel de Substancium pour le volet de la maintenance préventive. Au-delà des tables relationnelles classiques (utilisateurs, missions, sites), elle tire parti des capacités NoSQL de PostgreSQL en exploitant le type de données `JSONB` pour indexer et interroger efficacement les canevas de checklists et les diagnostics d'interventions associés.

3. Mécanisme et cycle de vie du mode "Offline-First"

Le fonctionnement en mode déconnecté repose sur un cycle de vie strict de la donnée, garantissant qu'aucune perte d'information ne survienne, même en cas de coupure réseau prolongée :

- **La Phase de Provisionnement (Online - Matin) :** Avant de démarrer sa tournée, le technicien doit initier une synchronisation descendante depuis un lieu disposant d'une connectivité internet (3G/4G/Wi-Fi). L'application mobile interroge l'API centrale pour récupérer le "pack mission" du jour. Ce pack contient l'inventaire théorique du site cible et les modèles JSON exacts des checklists. Ces données sont immédiatement écrites dans le SQLite local du téléphone, rendant l'application 100 % autonome.
- **La Phase d'Isolation Opérationnelle (Offline - Sur site) :** Une fois dans les salles de serveurs ou les data centers (zones blanches sans réseau), l'application mobile coupe ses requêtes vers l'extérieur et se focalise sur le stockage SQLite local. Lorsque le technicien saisit l'identifiant ou le numéro de série d'un équipement, la recherche s'effectue en quelques millisecondes dans la base locale. Le moteur de rendu de formulaires lit le schéma JSON en cache pour afficher dynamiquement l'interface d'audit. Chaque validation de diagnostic génère une écriture immédiate dans le SQLite local avec un drapeau (flag) applicatif `syncEnAttente = true`.
- **La Phase de Réconciliation Asynchrone (Online - Dès détection du réseau) :** L'application mobile intègre un écouteur de réseau (Network Listener) en tâche de fond. Dès qu'une connexion internet stable est rétablie (à la sortie du site client ou en fin de journée), l'application initie une synchronisation ascendante. Elle extrait de SQLite toutes les lignes marquées `syncEnAttente = true` et les transmet via une requête groupée (Bulk POST) à l'API REST. Le serveur API valide la cohérence des données, met à jour la base PostgreSQL centrale, puis renvoie un accusé de réception positif au mobile. À la réception de cet accord, l'application mobile met à jour ses indicateurs locaux à `syncEnAttente = false`, clôturant ainsi le cycle de réconciliation sans aucun risque de doublon.

II. Choix technologiques (La Stack)

La sélection des technologies repose sur les contraintes majeures du projet : la nécessité d'un fonctionnement hors-ligne fiable, la génération d'interfaces dynamiques, et le besoin d'une architecture centralisée robuste.

1. Front-end Mobile : React Native et SQLite

- **React Native (Application Mobile) :** Ce framework permet de développer une application multiplateforme (iOS et Android) à partir d'une base de code unique, garantissant une maintenance simplifiée. Son architecture basée sur des composants est idéale pour notre moteur de rendu dynamique, permettant d'instancier les interfaces (champs, cases à cocher) à la volée à partir des schémas JSON.
- **SQLite (Base de données embarquée) :** Pour garantir le paradigme "Offline-First", SQLite agit comme un cache local de haute disponibilité. Il permet de stocker les plannings et d'enregistrer les interventions en l'absence totale de réseau, offrant des temps de réponse instantanés au technicien sur le terrain.

2. Back-end : API REST avec FastAPI

- **Performance et Asynchronisme :** FastAPI est un framework Python ultra-rapide et nativement conçu pour l'asynchrone. Cette caractéristique est cruciale pour traiter simultanément les requêtes massives de synchronisation (Bulk POST) lorsque plusieurs techniciens retrouvent du réseau en fin de journée.
- **Validation native avec Pydantic :** FastAPI intègre Pydantic, un outil puissant qui validera automatiquement la structure des réponses JSON envoyées par l'application mobile avant même qu'elles n'atteignent la base de données, garantissant ainsi l'intégrité des formulaires.
- **Documentation automatique :** Le framework génère dynamiquement la documentation de l'API (Swagger/OpenAPI), facilitant l'intégration et les tests entre les environnements front-end et back-end.

3. Base de données centrale : PostgreSQL (Hébergement Cloud)

- **Fiabilité relationnelle** : En tant que cœur du système d'information, PostgreSQL garantit la conformité ACID (Atomicité, Cohérence, Isolation, Durabilité). Il gérera de manière robuste et intègre les relations complexes entre les utilisateurs, les marchés, les sites et l'inventaire matériel.
- **Flexibilité NoSQL avec le type JSONB** : C'est la justification clé de cette stack. PostgreSQL intègre nativement le type JSONB (JSON Binary). Ce format permet au serveur d'indexer les données à l'intérieur même des objets JSON. Le superviseur pourra ainsi exécuter des requêtes SQL performantes directement sur les réponses des formulaires dynamiques (par exemple, filtrer toutes les interventions où une valeur spécifique a été saisie), alliant ainsi la rigueur du relationnel à la flexibilité du NoSQL.
- **Hébergement externalisé (Cloud RDS)** : Pour assurer une haute disponibilité et s'affranchir des contraintes de maintenance matérielle, la base de données sera hébergée sur un service Cloud spécialisé (par exemple via AWS RDS, Render, ou Supabase). Cela garantira des sauvegardes automatisées et une scalabilité optimale au fur et à mesure de la croissance des données de Substancium.

CHAPITRE IV : GESTION DE PROJET ET PLANIFICATION

La réussite de la numérisation du processus de maintenance préventive de Substancium repose sur une organisation rigoureuse. Ce chapitre détaille la méthodologie adoptée, le découpage des tâches et l'échéancier prévisionnel du projet.

I. Méthodologie de travail : Approche Agile (Scrum)

Compte tenu de la complexité technique du projet (mode hors-ligne, moteur de formulaires dynamiques), le modèle de développement classique en "Cascade" a été écarté au profit de la méthode **Agile Scrum**. Cette approche itérative et incrémentale permet de :

- S'adapter rapidement aux éventuels retours et ajustements demandés par les encadrants de Substancium.
- Livrer des fonctionnalités testables régulièrement (à la fin de chaque itération).
- Identifier et résoudre très tôt les blocages techniques (notamment sur la synchronisation SQLite/PostgreSQL).

Rôles et organisation

- **Product Owner** : Le superviseur / encadrant chez Substancium (définit les priorités, valide les processus métier et les modèles JSON).
- **Équipe de développement** : L'ingénieur en charge de la conception, du développement (Front-end et Back-end) et des tests.
- **Cérémonies** : Des réunions de suivi hebdomadaires seront mises en place pour valider l'avancement, présenter les démonstrations des modules terminés et planifier l'itération suivante.

II. Découpage du projet en Sprints (Phases de réalisation)

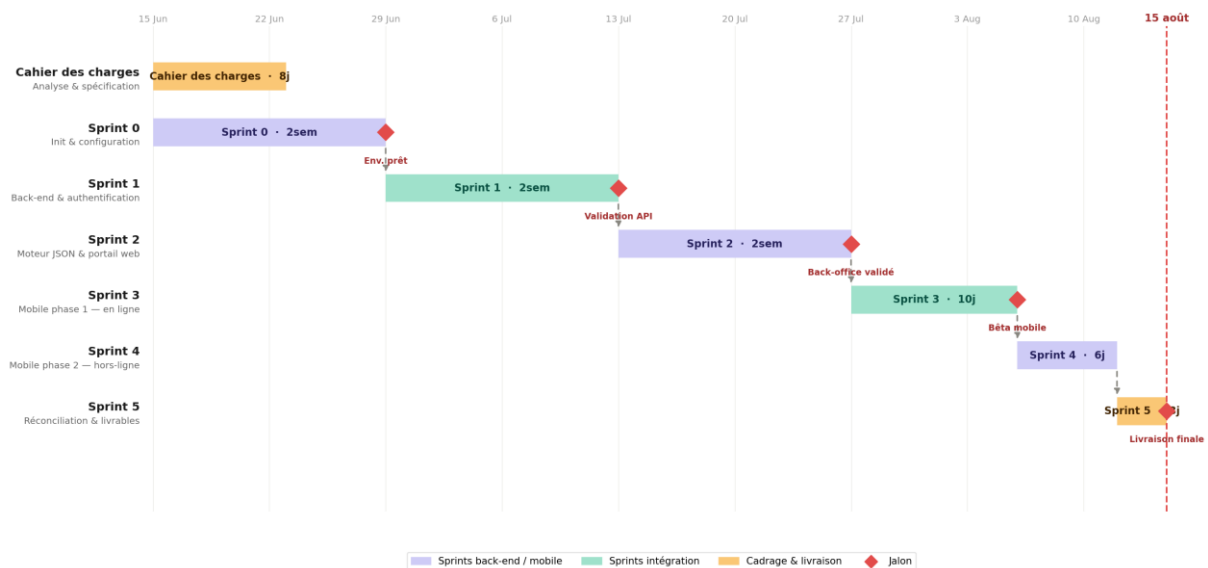


FIGURE 5: DIAGRAMME DE GANTT DE LA PLANIFICATION DES SPRINTS (METHODOLOGIE AGILE SCRUM)

Le cycle de développement est découpé en plusieurs itérations (Sprints) d'une durée moyenne de 2 à 3 semaines, structurées de manière chronologique et logique par rapport à l'architecture.

- **Sprint 0 : Initialisation et Configuration (Setup)**

- Mise en place de l'environnement de développement.
- Création et configuration de la base de données PostgreSQL (Hébergement Cloud).
- Initialisation du projet FastAPI et de l'application React Native.
- Mise en place du versioning (Git/GitHub) et de la structure des dossiers.

- **Sprint 1 : Socle Back-end et Authentification**

- Modélisation de la base de données relationnelle via l'ORM (Tables Utilisateurs, Marchés, Sites, Équipements).
- Développement du module d'authentification sécurisé (Génération et validation des tokens JWT).
- Création des endpoints (routes API) pour la gestion des référentiels (CRUD Marchés et Sites).

- **Sprint 2 : Le Moteur JSON et l'Interface Web**

- Développement du back-office Web (Dashboard) pour le Superviseur.
- Implémentation de la logique de planification des missions.
- Intégration du système d'importation et de validation des "JSON Schemas" (modèles de checklists) sur le serveur.

- **Sprint 3 : Application Mobile (Phase 1 - Mode En ligne)**
 - Développement de l'interface de connexion mobile.
 - Implémentation de la "Synchronisation Matinale" : téléchargement du planning, de l'inventaire et des schémas JSON via l'API.
 - Configuration de SQLite pour la persistance locale de ces données téléchargées.
- **Sprint 4 : Cœur de métier Mobile (Phase 2 - Mode Hors-ligne)**
 - Développement du moteur de rendu dynamique React Native : génération de l'interface UI d'audit à partir du schéma JSON stocké dans SQLite.
 - Logique de recherche matérielle (surbrillance verte si existant, ajout local si manquant).
 - Enregistrement local des diagnostics (interventions) et modification des statuts de mission.
- **Sprint 5 : Réconciliation et Livrables (Phase finale)**
 - Développement du service en tâche de fond (Background Task) pour la détection du réseau.
 - Implémentation de la "Synchronisation Finale" (envoi massif et asynchrone des données vers FastAPI).
 - Développement du module d'exportation PDF/Excel pour le client et le superviseur.
 - Tests d'intégration globaux, corrections de bugs, et rédaction de la documentation finale.

III. Livrables attendus

À l'issue de la période de réalisation, les éléments suivants seront remis à Substancium :

1. **L'Application Mobile Android/iOS** : Le fichier binaire compilé (.apk / .ipa) prêt à être déployé sur les terminaux des techniciens.
2. **Le Portail Web de Supervision** : Déployé et fonctionnel pour l'administration des missions.
3. **Le Code Source Complet** : L'intégralité des dépôts (Front-end Mobile, Front-end Web, et Back-end API) documentés.
4. **Le Dossier de Conception et d'Architecture** : Le présent document finalisé.
5. **Le Manuel Utilisateur** : Un guide d'utilisation simplifié à destination des superviseurs (pour la planification) et des techniciens (pour la manipulation de l'application sur site).

Conclusion Générale

La conception de cette solution marque une étape décisive dans la modernisation de la maintenance préventive chez Substancium Business Services (SBS). L'élaboration de ce cahier des charges a permis de transformer une problématique terrain complexe — la gestion archaïque des audits papier — en une architecture logicielle moderne, robuste et évolutive.

Le système proposé répond directement aux défis opérationnels grâce à des partis pris technologiques forts :

- **Continuité de service** : L'architecture "Offline-First" couplée à SQLite assure aux techniciens une autonomie totale dans les environnements critiques (data centers) dépourvus de réseau.
- **Flexibilité métier** : Le moteur de rendu JSON permet de modifier dynamiquement les checklists d'inspection depuis le serveur, sans mise à jour fastidieuse de l'application mobile.
- **Fiabilité des données** : La suppression de la double saisie et la synchronisation vers l'API centrale (FastAPI) et la base de données (PostgreSQL) garantissent une traçabilité exacte des interventions.

Au-delà de la résolution des contraintes actuelles, ce socle architectural ouvre de vastes perspectives d'évolution pour le système d'information de l'entreprise :

- **Business Intelligence** : L'exploitation des données via des outils décisionnels (ex: Power BI) pour analyser les pannes et optimiser les budgets informatiques.
- **Maintenance prédictive** : L'intégration future de modèles d'Intelligence Artificielle pour analyser l'historique des interventions et anticiper les défaillances.
- **Interopérabilité étendue** : L'interfaçage de l'API avec les solutions ERP existantes chez SBS ou ses clients.

En définitive, ce projet dépasse la simple dématérialisation d'un processus : il dote SBS d'un véritable outil stratégique pour consolider son expertise technique et offrir une qualité de service optimale à ses partenaires.